- (CAS) Programación en Java desde C#
  - Java y C#
    - **Similitudes**
    - **Diferencias**
  - 2. Estructuras de control
    - `if-else`
      - `switch`
      - `switch` **mejorado (C#)**
      - `for`
      - `foreach` **(o** `for` **mejorado)**
      - `while`
      - `do-while`
    - 3. Ejemplo de clases simples
    - 4. Ejemplo de herencia
    - 5. Ejemplo de clases abstractas y herencia
    - 6. Ejemplo de interfaces e implementación
    - Actividades
      - 1. **Estructuras de control**
        - **Condicionales (if/else, switch)**
        - **Bucles (for, while, do-while)**
      - 2. **Funciones**
      - 3. **Funciones recursivas**
  - Tabla de equivalencias entre C# y Java
- (VAL) Programació en Java des de C#
  - Java i C#
    - **Similituds**
    - **Diferències**
  - 2. Estructures de control
    - `if-else`
      - `switch`
      - `switch` **millorat (C#)**
      - `for`
      - `foreach` **(o** `for` **millorat)**
      - `while`
      - `do-while`
    - 3. Exemple de classes simples
    - 4. Exemple d'herència
    - 5. Exemple de

(CAS) Programación en Java desde C#

Java y C#

Java y C# son dos lenguajes de programación orientados a objetos con muchas similitudes, pero también con algunas diferencias clave, tanto a nivel técnico como filosófico. Antes de comenzar a aprender Java, vamos a repasar algunos puntos clave para facilitar nuestra adaptación:

Similitudes

1. **Sintaxis**: La sintaxis básica de ambos lenguajes es muy parecida, ya que C# fue diseñado influenciado por Java. Esto hace que moverse entre uno y otro sea relativamente sencillo. Ambos usan `{}` para definir bloques de código y `;` para terminar sentencias.

2. **Orientación a objetos**: Ambos son lenguajes puramente orientados a objetos, con clases, herencia, polimorfismo, encapsulamiento, etc.

3. **Recolección de basura (Garbage Collection)**: Tanto Java como C# cuentan con sistemas automáticos de recolección de basura que se encargan de la gestión de la memoria, lo que simplifica el desarrollo y evita fugas de memoria.

4. **Entornos virtuales**: Java corre sobre la **Java Virtual Machine (JVM)**, mientras que C# corre sobre el **Common Language Runtime (CLR)**. Ambos se ejecutan sobre máquinas virtuales, lo que proporciona independencia del hardware subyacente y facilita la portabilidad.

5. **Librerías y frameworks**: Ambos lenguajes tienen bibliotecas y frameworks amplios que cubren desde aplicaciones web hasta aplicaciones de escritorio y móviles (Spring para Java, ASP.NET para C#).

Diferencias

1. **Plataformas**:

    - **Java**: Es multiplataforma por naturaleza. Su lema es "escribe una vez, corre en cualquier lugar", ya que cualquier sistema con una JVM puede ejecutar código Java.
    - **C#**: Originalmente estaba más ligado a **Windows** y al ecosistema de Microsoft, aunque hoy en día con .NET Core y .NET 5/6 es más multiplataforma.

2. **Propiedad y Comunidad**:

    - **Java**: Es propiedad de Oracle y tiene una comunidad muy grande, con muchas contribuciones de código abierto.

- **C#**: Es desarrollado y mantenido por Microsoft. Aunque originalmente era más cerrado, .NET Core ha abierto más el ecosistema y la comunidad.

3. **Ecosistema de desarrollo**:
   - **Java**: Su ecosistema tiende a ser más modular, utilizando herramientas como **Maven** o **Gradle** para la gestión de dependencias. IDEs populares incluyen **IntelliJ IDEA** y **Eclipse**.
   - **C#**: Tiene una integración más fuerte en el ecosistema de **Visual Studio**, lo que permite una experiencia de desarrollo muy pulida y productiva dentro de este IDE. **NuGet** es su principal herramienta de gestión de paquetes.

4. **Tipos de datos y punteros**:
   - **Java**: No permite el uso directo de punteros ni acceso a la memoria fuera de sus reglas de seguridad.
   - **C#**: Aunque generalmente no se usan, permite el uso de punteros bajo el contexto de código "unsafe" (inseguro), algo que Java no tiene.
   - C# permite usar valores por referencia en los parámetros de las funciones, mientras que en Java dependerá exclusivamente del tipo de datos.

2. Estructuras de control

Vamos a repasar cómo son las estructuras de control en C# y cuál es su equivalente en Java.

if-else

C#:

```
int x = 10;
if (x > 5) {
    Console.WriteLine("Mayor a 5");
} else {
    Console.WriteLine("Menor o igual a 5");
}
```

Java:

```
int x = 10;
if (x > 5) {
    System.out.println("Mayor a 5");
} else {
    System.out.println("Menor o igual a 5");
}
```

`switch`

C#:

```
int day = 3;
switch (day) {
    case 1:
        Console.WriteLine("Lunes");
        break;
    case 2:
        Console.WriteLine("Martes");
        break;
    default:
        Console.WriteLine("Otro día");
        break;
}
```

Java:

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    default:
        System.out.println("Otro día");
        break;
}
```

`switch` mejorado (C#)

C#:

```
string dayName = day switch
{
    1 => "Lunes",
    2 => "Martes",
    _ => "Otro día"
};
Console.WriteLine(dayName);
```

Java (con Java 14 o superior):

```
String dayName = switch (day) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    default -> "Otro día";
};
System.out.println(dayName);
```

`for`

C#:

```
for (int i = 0; i < 5; i++) {
    Console.WriteLine(i);
}
```

Java:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

`foreach` (o `for` mejorado)

C#:

```
int[] numbers = { 1, 2, 3, 4, 5 };
foreach (var num in numbers) {
    Console.WriteLine(num);
}
```

Java:

```
int[] numbers = { 1, 2, 3, 4, 5 };
for (int num : numbers) {
    System.out.println(num);
}
```

`while`

C#:

```
int i = 0;
while (i < 5) {
    Console.WriteLine(i);
    i++;
}
```

Java:

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

do-while

C#:

```
int i = 0;
do {
    Console.WriteLine(i);
    i++;
} while (i < 5);
```

Java:

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

Como se puede observar, es casi siempre igual salvo en las formas "mejoradas" de hacer algunas cosas. También es notable el uso de `System.out` en java, en lugar de `Console`.

---

3. Ejemplo de clases simples

C#:

```
public class Persona {
    public string Nombre { get; set; }
```

```
    public int Edad { get; set; }

    public Persona(string nombre, int edad) {
        Nombre = nombre;
        Edad = edad;
    }

    public void Saludar() {
        Console.WriteLine($"Hola, mi nombre es {Nombre} y tengo {Edad}
años.");
    }
}
```

Java:

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void saludar() {
        System.out.println("Hola, mi nombre es " + nombre + " y tengo " +
edad + " años.");
    }

    // Getters y setters en Java
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        this.edad = edad;
    }
}
```

Detalles a tener en cuenta son la forma de trabajar con getters y setters y también que en Java los métodos y los atributos comienzan en minúscula, mientras que en C# los métodos

comienzan en mayúscula. Con IntelliJ, podemos generar los constructores, getters y setters de forma automática, así como otros métodos (click derecho, generar métodos).

---

4. Ejemplo de herencia

C#:

```
public class Animal {
    public void HacerSonido() {
        Console.WriteLine("Sonido genérico");
    }
}

public class Perro : Animal {
    public void Ladrar() {
        Console.WriteLine("Guau!");
    }
}
```

Java:

```
public class Animal {
    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

public class Perro extends Animal {
    public void ladrar() {
        System.out.println("Guau!");
    }
}
```

En el caso de la herencia, la diferencia es clara. C# maneja la herencia como lo hace C++, mientras que Java utiliza palabras clave específicas para especificar el tipo de herencia.

---

5. Ejemplo de clases abstractas y herencia

C#:

```
public abstract class Forma {
    public abstract void Dibujar();
}

public class Circulo : Forma {
```

```
    public override void Dibujar() {
        Console.WriteLine("Dibujando un círculo");
    }
}
```

Java:

```
public abstract class Forma {
    public abstract void dibujar();
}

public class Circulo extends Forma {
    @Override
    public void dibujar() {
        System.out.println("Dibujando un círculo");
    }
}
```

---

6. Ejemplo de interfaces e implementación

C#:

```
public interface IVolador {
    void Volar();
}

public class Pajaro : IVolador {
    public void Volar() {
        Console.WriteLine("El pájaro está volando");
    }
}
```

Java:

```
public interface Volador {
    void volar();
}

public class Pajaro implements Volador {
    @Override
    public void volar() {
        System.out.println("El pájaro está volando");
    }
}
```

Como se ve, C# trata la implementación de interfaces como un tipo de herencia más, mientras que Java usa una palabra clave diferente. En ambos lenguajes, una clase solo puede heredar de otra clase, pero puede implementar tantas interfaces como necesite.

Actividades

A continuación, 18 ejercicios para aprender a programar en Java. Puedes consultar estos apuntes (apartados 1, 2, 3 y 4) o la documentación oficial de Java si lo necesitas.

1. Estructuras de control

Condicionales (if/else, switch)

1. **Mayor de dos números:** Escribe un programa que reciba dos números por teclado e imprima cuál de ellos es el mayor. Si son iguales, debe indicarlo.

2. **Par o impar:** Crea un programa que pida un número entero y determine si es par o impar.

3. **Calificación:** Escribe un programa que reciba una calificación numérica (0-100) e imprima la correspondiente letra según el sistema educativo:

   - A (90-100)
   - B (80-89)
   - C (70-79)
   - D (60-69)
   - F (0-59)

4. **Días de la semana:** Escribe un programa que reciba un número del 1 al 7 e imprima el día de la semana correspondiente (por ejemplo, 1 es Lunes, 2 es Martes, etc.). Usa una estructura `switch`.

Bucles (for, while, do-while)

5. **Suma de los primeros N números:** Escribe un programa que pida un número entero N y calcule la suma de los números desde 1 hasta N.

6. **Tablas de multiplicar:** Crea un programa que pida un número entero entre 1 y 10 y muestre su tabla de multiplicar completa.

7. **Números primos:** Escribe un programa que pida un número N y muestre los números primos entre 1 y N.

8. **Número de dígitos:** Escribe un programa que reciba un número entero y determine cuántos dígitos tiene. Usa un bucle `while`.

---

2. Funciones

9. **Función para determinar si un número es primo:** Crea una función llamada `esPrimo` que reciba un número entero y retorne `true` si el número es primo y `false` en caso contrario. Usa esta función en el programa principal para imprimir si un número ingresado es primo o no.

10. **Función para calcular el factorial:** Escribe una función llamada `factorial` que reciba un número entero y retorne su factorial (n!). Usa un bucle para implementarla.

11. **Función que determine si un número es palíndromo:** Crea una función llamada `esPalindromo` que reciba un número entero y retorne `true` si el número es un palíndromo (se lee igual de izquierda a derecha y de derecha a izquierda).

12. **Máximo común divisor:** Escribe una función llamada `mcd` que reciba dos números y retorne su máximo común divisor utilizando el algoritmo de Euclides.

13. **Suma de dígitos:** Crea una función llamada `sumaDigitos` que reciba un número entero y retorne la suma de sus dígitos.

---

3. Funciones recursivas

14. **Factorial recursivo:** Reescribe la función `factorial` del ejercicio anterior utilizando recursividad.

15. **Fibonacci recursivo:** Escribe una función recursiva llamada `fibonacci` que reciba un número entero N y retorne el N-ésimo número de la secuencia de Fibonacci.

16. **Suma de números hasta N recursivo:** Crea una función recursiva llamada `sumaRecursiva` que reciba un número N y retorne la suma de los números desde 1 hasta N.

17. **Potencia recursiva:** Escribe una función recursiva llamada `potencia` que reciba dos enteros, `base` y `exponente`, y retorne el valor de base^exponente sin utilizar operadores de potenciación.

18. **Inversión de una cadena:** Crea una función recursiva llamada `invertirCadena` que reciba una cadena de texto y la devuelva invertida (por ejemplo, "Hola" -> "aloH").

Tabla de equivalencias entre C# y Java

Concepto	C#	Java
Declaración de clase	`class MyClass {}`	`class MyClass {}`
Declaración de método	`public void MyMethod()`	`public void myMethod()`
Constructor	`public MyClass() {}`	`public MyClass() {}`

Concepto	C#	Java
Modificadores de acceso	`public`, `private`, `protected`	`public`, `private`, `protected`
Propiedades automáticas	`public int MyProp { get; set; }`	Se usa directamente con métodos getter y setter manuales
Declaración de array	`int[] arr = new int[5];`	`int[] arr = new int[5];`
Arreglos dinámicos (List)	`List<int> list = new List<int>();`	`List<Integer> list = new ArrayList<>();`
Herencia	`class MyClass : ParentClass {}`	`class MyClass extends ParentClass {}`
Interfaces	`interface IMyInterface {}`	`interface MyInterface {}`
Implementación de interfaz	`class MyClass : IMyInterface {}`	`class MyClass implements MyInterface {}`
Estructura `if-else`	Igual en ambos lenguajes	Igual en ambos lenguajes
Estructura `switch`	`switch(exp) { case 1: ... }`	`switch(exp) { case 1: ... }`
Ciclo `for`	`for (int i = 0; i < 10; i++)`	`for (int i = 0; i < 10; i++)`
Ciclo `foreach`	`foreach (var item in collection)`	`for (Type item : collection)`

(VAL) Programació en Java des de C#

Java i C#

Java i C# són dos llenguatges de programació orientats a objectes amb moltes similituds, però també amb algunes diferències clau, tant a nivell tècnic com filosòfic. Abans de començar a aprendre Java, repassarem alguns punts clau per a facilitar la nostra adaptació:

Similituds

1. **Sintaxi**: La sintaxi bàsica de tots dos llenguatges és molt semblant, ja que C# es va dissenyar influenciat per Java. Això fa que passar d'un a un altre siga relativament senzill. Tots dos utilitzen `{}` per a definir blocs de codi i `;` per a acabar les sentències.

2. **Orientació a objectes**: Tots dos són llenguatges purament orientats a objectes, amb classes, herència, polimorfisme, encapsulació, etc.

3. **Recollida d'escombraries (Garbage Collection)**: Tant Java com C# compten amb sistemes automàtics de recollida d'escombraries que s'encarreguen de la gestió de la memòria, simplificant el desenvolupament i evitant fugues de memòria.

4. **Entorns virtuals**: Java s'executa sobre la **Java Virtual Machine (JVM)**, mentre que C# s'executa sobre el **Common Language Runtime (CLR)**. Tots dos s'executen sobre màquines virtuals, la qual cosa proporciona independència de maquinari i facilita la portabilitat.

5. **Llibreries i frameworks**: Tots dos llenguatges tenen biblioteques i frameworks amplis que cobreixen des d'aplicacions web fins a aplicacions d'escriptori i mòbils (Spring per a Java, ASP.NET per a C#).

Diferències

1. **Plataformes**:

   - **Java**: És multiplataforma per naturalesa. El seu lema és "escriu una vegada, executa en qualsevol lloc", ja que qualsevol sistema amb una JVM pot executar codi Java.
   - **C#**: Originalment estava més lligat a **Windows** i a l'ecosistema de Microsoft, encara que avui dia, amb .NET Core i .NET 5/6, és més multiplataforma.

2. **Propietat i Comunitat**:

   - **Java**: És propietat d'Oracle i té una comunitat molt gran, amb moltes contribucions de codi obert.
   - **C#**: És desenvolupat i mantingut per Microsoft. Encara que originalment era més tancat, .NET Core ha obert més l'ecosistema i la comunitat.

3. **Ecosistema de desenvolupament**:
   - **Java**: El seu ecosistema tendeix a ser més modular, utilitzant eines com **Maven** o **Gradle** per a la gestió de dependències. IDEs populars inclouen **IntelliJ IDEA** i **Eclipse**.
   - **C#**: Té una integració més forta amb l'ecosistema de **Visual Studio**, que permet una experiència de desenvolupament molt polida i productiva dins d'aquest IDE. **NuGet** és la seua principal eina de gestió de paquets.
4. **Tipus de dades i punters**:
   - **Java**: No permet l'ús directe de punters ni l'accés a la memòria fora de les seues regles de seguretat.
   - **C#**: Encara que generalment no s'utilitzen, permet l'ús de punters en el context de codi "unsafe" (insegur), cosa que Java no permet.
   - C# permet utilitzar valors per referència en els paràmetres de les funcions, mentre que en Java dependrà exclusivament del tipus de dades.

2. Estructures de control

Vegem com són les estructures de control en C# i quin és el seu equivalent en Java.

if-else

C#:

```
int x = 10;
if (x > 5) {
    Console.WriteLine("Major a 5");
} else {
    Console.WriteLine("Menor o igual a 5");
}
```

Java:

```
int x = 10;
if (x > 5) {
    System.out.println("Major a 5");
} else {
    System.out.println("Menor o igual a 5");
}
```

switch

C#:

```
int day = 3;
switch (day) {
    case 1:
        Console.WriteLine("Dilluns");
        break;
    case 2:
        Console.WriteLine("Dimarts");
        break;
    default:
        Console.WriteLine("Un altre dia");
        break;
}
```

Java:

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Dilluns");
        break;
    case 2:
        System.out.println("Dimarts");
        break;
    default:
        System.out.println("Un altre dia");
        break;
}
```

`switch` millorat (C#)

C#:

```
string dayName = day switch
{
    1 => "Dilluns",
    2 => "Dimarts",
    _ => "Un altre dia"
};
Console.WriteLine(dayName);
```

Java (amb Java 14 o superior):

```
String dayName = switch (day) {
    case 1 -> "Dilluns";
    case 2 -> "Dimarts";
```

```
    default -> "Un altre dia";
};
System.out.println(dayName);
```

`for`

C#:

```
for (int i = 0; i < 5; i++) {
    Console.WriteLine(i);
}
```

Java:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

`foreach` (o `for` millorat)

C#:

```
int[] numbers = { 1, 2, 3, 4, 5 };
foreach (var num in numbers) {
    Console.WriteLine(num);
}
```

Java:

```
int[] numbers = { 1, 2, 3, 4, 5 };
for (int num : numbers) {
    System.out.println(num);
}
```

`while`

C#:

```
int i = 0;
while (i < 5) {
```

```
    Console.WriteLine(i);
    i++;
}
```

Java:

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

do-while

C#:

```
int i = 0;
do {
    Console.WriteLine(i);
    i++;
} while (i < 5);
```

Java:

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

Com es pot observar, quasi sempre és igual llevat de les formes "millorades" de fer algunes coses. També és notable l'ús de `System.out` en Java, en lloc de `Console`.

---

3. Exemple de classes simples

C#:

```
public class Persona {
    public string Nom { get; set; }
    public int Edat { get; set; }

    public Persona(string nom, int edat) {
```

```
        Nom = nom;
        Edat = edat;
    }

    public void Saludar() {
        Console.WriteLine($"Hola, el meu nom és {Nom} i tinc {Edat}
anys.");
    }
}
```

Java:

```
public class Persona {
    private String nom;
    private int edat;

    public Persona(String nom, int edat) {
        this.nom = nom;
        this.edat = edat;
    }

    public void saludar() {
        System.out.println("Hola, el meu nom és " + nom + " i tinc " + edat
+ " anys.");
    }

    // Getters i setters en Java
    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public int getEdat() {
        return edat;
    }

    public void setEdat(int edat) {
        this.edat = edat;
    }
}
```

---

4. Exemple d'herència

C#:

```
public class Animal {
    public void FerSo() {
        Console.WriteLine("So genèric");
    }
}

public class Gos : Animal {
    public void Bordar() {
        Console.WriteLine("Bup bup!");
    }
}
```

Java:

```
public class Animal {
    public void ferSo() {
        System.out.println("So genèric");
    }
}

public class Gos extends Animal {
    public void bordar() {
        System.out.println("Bup bup!");
    }
}
```

---

5. Exemple de

manipulació de llistes i altres col·leccions

C#:

```
List<int> numeros = new List<int> { 1, 2, 3, 4, 5 };
foreach (var numero in numeros) {
    Console.WriteLine(numero);
}
```

Java:

```
List<Integer> numeros = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
for (int numero : numeros) {
    System.out.println(numero);
}
```

Amb aquests exemples, pots començar a programar en Java amb una transició suau des del coneixement de C#. Les estructures són pràcticament les mateixes, i els conceptes de programació orientada a objectes continuen sent igual d'aplicables.